

CHAPTER 4

Switching, Multiplexing and Error Detection

Circuit, message and packet switching · datagram and virtual circuit · FDM, TDM, WDM · framing · parity, checksum, CRC · Hamming code

QUICK TRICK

Learn the fast tricks first (details below):

1. **Circuit** switching = dedicated path (telephone). **Packet** switching = data cut into packets (Internet). **Message** switching = whole message stored and forwarded.
2. **FDM** shares **frequency** (analog, TV/radio). **TDM** shares **time** (digital, telephone trunks). **WDM** shares **wavelength** (fibre).
3. **Parity** catches an **odd** number of bit errors only. **CRC** is the strongest. **Hamming** can **correct** one bit.
4. Detect up to $d - 1$ errors, correct up to $\lfloor (d - 1) / 2 \rfloor$ errors, where d = Hamming distance.

4.1 Switching: how data finds its way across many links

● LEVEL 1 · BEGINNER

The problem. A big network cannot join every computer to every other computer by a wire (that would be a full mesh, Chapter 1). **The idea.** Computers connect to **switching nodes**, and the nodes pass data from link to link until it reaches the destination. The way the nodes do this is called **switching**. There are three main techniques.

(a) Circuit switching

A **dedicated path** is first built from sender to receiver, then the data flows, then the path is released. Example: an ordinary **telephone call** on the PSTN.

Phase	What happens
1. Circuit establishment	Dialling: the network reserves a link on every hop from source to destination
2. Data transfer	Data (voice) flows continuously on the reserved path at a fixed rate; no delay at nodes
3. Circuit disconnect	Hang-up: the reserved links are released for others

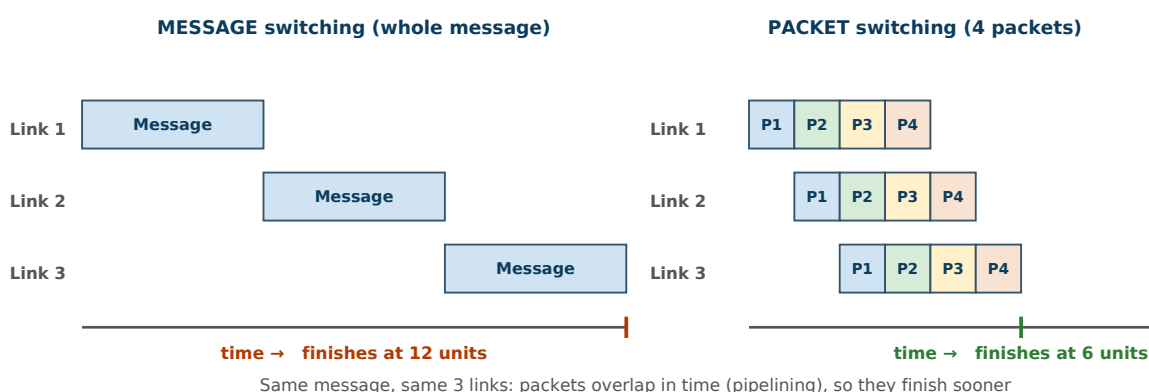
Good: fixed bandwidth, constant delay, data arrives in order, ideal for voice. **Bad:** setup time before talking; the line is wasted during silence (nobody else can use it); a call can be **blocked** (busy tone) if no free path; costly for bursty data.

(b) Message switching

The **whole message** (no size limit) is sent to the first node, which **stores** it completely and then **forwards** it to the next node (**store-and-forward**). There is no setup. Example: old telegram or e-mail relay. **Good:** no circuit setup, links shared. **Bad:** long delay at each node, nodes need large storage, one big message can block a link, not suitable for real-time use.

(c) Packet switching

The message is divided into small pieces called **packets**. Each packet carries a **header** (source and destination address, sequence number) and is sent separately. Nodes use store-and-forward **per packet**, so packets travel one behind the other and many users share the same links (**statistical multiplexing**). The Internet works this way.



Delay formulas (k links in a row, all with rate R , ignoring propagation and queuing; L = message size in bits):

Message switching: $T = k \times L \div R$

Packet switching with packets of size p ($N = L \div p$ packets): $T = (k + N - 1) \times p \div R$

Example. $L = 4000$ bits, $R = 1$ Mbps, $k = 3$ links, $p = 1000$ bits ($N = 4$).

Message switching: $T = 3 \times 4000 \div 10^6 = 12$ ms. Packet switching: $T = (3 + 4 - 1) \times 1000 \div 10^6 = 6$ ms. Packets overlap, so the same data arrives in half the time.

Two ways to do packet switching

Point	Datagram (connectionless)	Virtual circuit (connection-oriented)
Setup	None	A path is set up first (call request, accept); released at the end
Route	Each packet is routed independently	All packets follow the same path
Header	Full destination address in every packet	Short virtual circuit number (VC id) after setup
Order	May arrive out of order, may be lost	Arrive in order
Failure of a node	Other packets use another route	The circuit breaks and must be set up again
Examples	Internet (IP)	X.25, Frame Relay, ATM

Point	Circuit switching	Message switching	Packet switching
Path	Dedicated, set up first	None; hop by hop	Datagram: none; VC: set up
Unit of data	Continuous stream	Whole message	Small packets
Setup delay	Yes	No	Datagram: no; VC: yes
Bandwidth use	Wasted when idle	Shared	Shared, most efficient
Store-and-forward	No	Yes (whole message)	Yes (each packet)
Delay	Constant, low after setup	High, variable	Variable (queuing)
Best for	Voice calls	Old telegram, e-mail relay	Data, web, video over Internet
Example	Telephone (PSTN)	Telegram	Internet

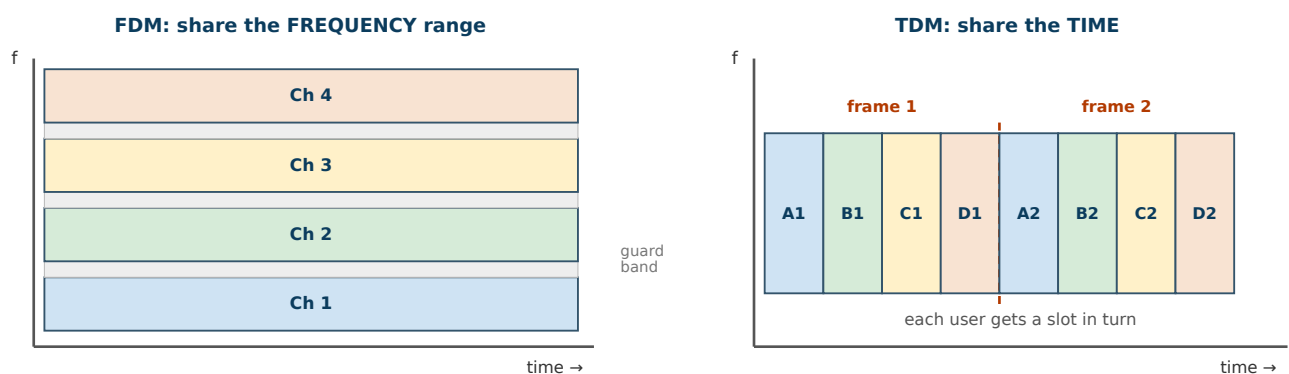
QUICK TRICK

Telephone = circuit. Internet = packet. Words to link: circuit $\rightarrow$ *dedicated, reserved*; packet $\rightarrow$ *header, statistical multiplexing, store-and-forward per packet*.

4.2 Multiplexing

● LEVEL 1 • BEGINNER

Multiplexing means sending many signals together on **one** shared link. A **MUX** (multiplexer) at the sender combines them and a **DEMUX** (demultiplexer) at the receiver separates them again. It saves cable: one thick link instead of many thin ones.



Type	Full form	Idea	Signal	Use
FDM	Frequency Division Multiplexing	Each channel gets its own frequency band; all send at the same time. Guard bands between bands avoid overlap.	Analog	Radio and TV broadcast, cable TV, old telephone trunks
TDM	Time Division Multiplexing	Each channel gets the whole link for a short time slot , in turns, repeating in frames.	Digital	Telephone trunks (T1/E1), SONET
WDM	Wavelength Division Multiplexing	Different colours (wavelengths) of light carry different channels on one fibre. DWDM packs many close wavelengths.	Optical	Backbone fibre links
CDM / CDMA	Code Division Multiple Access	All users use the same band at the same time; each has a unique code .	Digital / radio	Some mobile networks (2G/3G)

Types of TDM

Point	Synchronous TDM	Statistical (asynchronous) TDM
Slot assignment	Fixed slot for each input, in every frame	Slot given only to inputs that have data
Idle input	Its slot is sent empty (wasted)	No slot is wasted
Address in slot	Not needed (position tells the owner)	Needed (each slot carries the sender's address)
Efficiency	Lower for bursty data	Higher; link can be smaller than the sum of inputs
Complexity	Simple	More complex (buffers, addresses)

Example (FDM). 5 voice channels of 100 kHz each, with 4 guard bands of 10 kHz between them: bandwidth = $5 \times 100 + 4 \times 10 = 540 \text{ kHz}$.

Example (TDM). 4 sources of 1 Mbps each are combined by synchronous TDM. The link must carry at least $4 \times 1 = 4 \text{ Mbps}$ (more if framing bits are added).

Digital telephone lines: PCM, T1 and E1

A voice signal has frequencies up to about 4 kHz. By the **sampling theorem** (sampling rate $\geq 2 \times$ highest frequency) it is sampled **8000 times per second**, and each sample is coded in **8 bits**. This is **PCM** (Pulse Code Modulation): sampling $\rightarrow$ quantisation $\rightarrow$ encoding. One voice channel = $8000 \times 8 = 64 \text{ kbps}$.

Line	Channels	Calculation	Rate	Used in
T1	24	$24 \times 8 \text{ bits} + 1 \text{ framing bit} = 193 \text{ bits}$ every 125 μs	1.544 Mbps	North America, Japan
E1	32 (30 voice + 2 control)	$32 \times 64 \text{ kbps}$	2.048 Mbps	Europe, India and most countries

COMMON MISTAKE

- Mixing FDM and TDM: FDM channels send **together** on different frequencies; TDM channels send **in turn** on the same frequency.
- Saying TDM is for analog. TDM needs digital signals (analog voice is first digitised by PCM).
- Forgetting that FDM needs guard bands, so total bandwidth is more than the sum of the channels.

4.3 Framing: packing bits into frames

● LEVEL 2 • EXAM

The data link layer packs bits into units called **frames**: **header** (addresses, control) + **data** (payload) + **trailer** (error-check code such as CRC). The receiver must know where a frame starts and ends. Common methods: character count, **flag bytes**, and **flag bits with bit stuffing**.

Bit stuffing (used in HDLC and PPP-like protocols). The flag is **01111110**. To stop this pattern appearing inside the data, the sender inserts a **0 after every five consecutive 1s**. The receiver removes a 0 that follows five 1s.

Example. Data = 01111110 (looks like a flag). After stuffing: 0 1111 0 1 0 = **011111010**. The receiver sees five 1s followed by 0, removes that 0 and gets the original data back.

4.4 Errors in transmission

● LEVEL 1 · BEGINNER

Noise (Chapter 2) can change a 0 into a 1 or a 1 into a 0. We use **extra bits (redundancy)** added by the sender to find (and sometimes fix) such errors.

Error type	Meaning
Single-bit error	Only one bit of the data unit is changed
Burst error	Two or more bits are changed. The burst length is counted from the first changed bit to the last changed bit (the bits in between may be correct)

Approach	Meaning	Methods
Error detection	Only find that an error happened; ask the sender to send again	Parity, 2-D parity, checksum, CRC
Error correction	Find the exact bit and fix it at the receiver (needs more redundancy)	Hamming code

Simple parity check

One extra bit, the **parity bit**, is added so that the total number of 1s is **even** (even parity) or **odd** (odd parity). The receiver counts the 1s again.

Example. Data 1011001 has four 1s. Even parity bit = **0** → sent 1011001**0**. Odd parity bit = **1** → sent 1011001**1**. If the receiver (even parity) gets 10110011, it counts five 1s (odd), so an **error is detected**.

BPSC POINT

Simple parity **detects every odd number of bit errors** (1, 3, 5 ...) but **misses every even number** (2, 4 ...), because the count stays even. It cannot correct and cannot locate the error.

Two-dimensional (2-D) parity

Data bits are arranged in rows and columns. A parity bit is added for **each row** and for **each column** (and one for the parity row/column). A single-bit error makes exactly one row and one column fail; their **intersection** is the wrong bit, so it can be **corrected**.

	C1	C2	C3	C4	Row parity
R1	1	0	1	0	0
R2	1	1	0	0	0
R3	0	1	1	1	1
Column parity	0	0	0	1	1

If bit R2-C3 flips from 0 to 1, row R2 and column C3 both show a parity failure, which points to R2-C3. Flip it back to correct it.

Checksum

The data is split into equal-size words (for example 16 bits in the Internet). The sender **adds all words** using **one's complement addition** (any carry out of the top is added back at the bottom, called wrap-around), then takes the **complement** of the sum. This is the **checksum** and it is sent with the data. The receiver adds all words including the checksum; if the complement of the result is **0**, the data is accepted.

Example (4-bit words). Send 7, 11, 12, 0, 6. Sum = 36 = 100100 (binary). Wrap the two extra left bits: 0100 + 10 = 0110 = 6. Checksum = complement of 6 = 15 - 6 = 9.

Receiver: 7 + 11 + 12 + 0 + 6 + 9 = 45 = 101101 → 1101 + 10 = 1111 = 15 → complement = **0** → data is correct.

Cyclic Redundancy Check (CRC)

The most powerful and widely used method (Ethernet, Wi-Fi, ZIP). The data is treated as a **polynomial** and divided (using modulo-2 arithmetic, which is XOR with no carries) by an agreed **generator polynomial**. The **remainder** is the CRC.

Step	Rule
1	Generator has $n + 1$ bits (degree n). Append n zeros to the data.
2	Divide by the generator using XOR division (modulo-2).
3	The n -bit remainder replaces the n zeros. Send data + remainder.
4	Receiver divides the received word by the same generator. Remainder 0 = no error .

Example. Data = 1001, generator = 1011 ($x^3 + x + 1$, so $n = 3$). Dividend = 1001**000**.

Step	Bits taken	Action	Result (remainder so far)
1	1001	Leading 1: XOR with 1011	0010 → keep 010
2	010 + next bit 0 = 0100	Leading 0: no XOR	100
3	100 + next bit 0 = 1000	Leading 1: XOR with 1011	0011 → keep 011
4	011 + next bit 0 = 0110	Leading 0: no XOR	110

CRC = **110**. Sent codeword = 1001**110**. Check at the receiver: $1001110 \div 1011$ gives remainder 000, so it is accepted.

QUICK TRICK

CRC remainder shortcut: work with polynomials. Data $x^3 + 1$ times $x^3 = x^6 + x^3$. With $x^3 = x + 1$: $x^6 = (x + 1)^2 = x^2 + 1$, so remainder = $x^2 + 1 + x + 1 = x^2 + x = \mathbf{110}$ ✓.

Method	Extra bits	Detects	Corrects	Weakness / note
Simple parity	1	All odd-bit errors	No	Misses even-bit errors
2-D parity	Row + column bits	All 1, 2, 3-bit errors and most bursts	Single bit	More overhead
Checksum	16 bits (Internet)	Most errors	No	Weaker than CRC; used in IP, TCP, UDP headers
CRC	n bits (CRC-32 uses 32)	All single-bit, all odd, all bursts up to n bits, most longer bursts	No	Best detection; used in Ethernet and Wi-Fi
Hamming code	r check bits	Up to 2 bits	Single bit	More bits; used in memory (ECC)

Hamming distance and Hamming code

Hamming distance between two equal-length words = number of positions in which they differ. Example: 10110 and 11010 differ in 2 places, so the distance is 2. **Minimum distance** $d_{\min}$ of a code decides its power:

Goal	Condition
Detect up to s errors	$d_{\min} \geq s + 1$ (detect $d - 1$)
Correct up to t errors	$d_{\min} \geq 2t + 1$ (correct $\lfloor (d - 1) / 2 \rfloor$)

Example: the code {000, 111} has $d = 3$, so it detects 2 errors and corrects 1 error.

Hamming code adds r check (parity) bits to m data bits so that a **single-bit error** can be located and corrected. The condition is $2^r \geq m + r + 1$. Total bits $n = m + r$. Check bits sit at positions that are powers of 2 (1, 2, 4, 8 ...).

Data bits m	4	7	8	11
Check bits r	3	4	4	4
Total $n = m + r$	7	11	12	15

Worked example, Hamming (7,4). Positions: 1 = p_1 , 2 = p_2 , 3 = d_1 , 4 = p_4 , 5 = d_2 , 6 = d_3 , 7 = d_4 . Data $d_1 d_2 d_3 d_4 = 1 0 1 1$.

p_1 checks positions 1, 3, 5, 7 → $d_1, d_2, d_4 = 1, 0, 1$ → two 1s, so $p_1 = \mathbf{0}$.

p_2 checks positions 2, 3, 6, 7 → $d_1, d_3, d_4 = 1, 1, 1$ → three 1s, so $p_2 = \mathbf{1}$.

p_4 checks positions 4, 5, 6, 7 → $d_2, d_3, d_4 = 0, 1, 1$ → two 1s, so $p_4 = \mathbf{0}$.

Codeword (positions 1 to 7) = **0 1 1 0 0 1 1**.

Finding an error. Suppose position 6 flips and 0110001 is received. Recompute the three checks (even parity): group (1, 3, 5, 7) = 0,1,0,1 → OK, $c_1 = 0$; group (2, 3, 6, 7) = 1,1,0,1 → fail, $c_2 = 1$; group (4, 5, 6, 7) = 0,0,0,1 → fail, $c_4 = 1$. Syndrome $c_4 c_2 c_1 = 110 = 6$, so **position 6 is wrong**. Flip it to correct.

Error control by retransmission (ARQ) in brief

When an error is only **detected**, the receiver does not send an acknowledgement (ACK) or sends a negative one (NAK), and the sender transmits the frame again. This is **Automatic Repeat reQuest (ARQ)**.

Protocol	Idea	Window size (m-bit sequence number)
Stop-and-Wait	Send one frame, wait for its ACK, then send the next. Simple but slow on long links.	Sender 1, receiver 1
Go-Back-N	Send up to N frames without waiting; if one is lost or bad, resend it and all after it .	Sender $2^m - 1$, receiver 1
Selective Repeat	Resend only the lost or bad frame; the receiver buffers out-of-order frames.	Sender 2^{m-1} , receiver 2^{m-1}

4.5 Common mistakes

● LEVEL 2 · EXAM

COMMON MISTAKE

- Saying packet switching needs a dedicated path. Only **circuit** switching (and virtual-circuit setup) reserves a path; a datagram does not.
- Saying simple parity finds all errors. It misses **even** numbers of errors.
- Putting the parity bit wrongly: **even parity** makes the total number of 1s **even**.
- Checksum: forgetting the wrap-around carry or forgetting to take the **complement**.
- CRC: using ordinary subtraction. It is **XOR** (modulo-2); there are no borrows or carries.
- Using Hamming distance d as the number of errors. Detect **d - 1**, correct **[(d - 1) / 2]**.
- Confusing switching (path through the network) with multiplexing (sharing one link).

Quick-Solve Tricks — Summary

Situation	Trick
Telephone network	Circuit switching: setup → transfer → release
Internet	Packet switching (datagram); X.25, Frame Relay, ATM = virtual circuit
Message vs packet time	Message: $k \cdot L/R$ · Packet: $(k + N - 1) \cdot p/R$
FDM / TDM / WDM	Frequency (analog) / Time (digital) / Wavelength (fibre)
FDM bandwidth	Sum of channels + guard bands
Voice digitising	$8000 \text{ samples/s} \times 8 \text{ bits} = 64 \text{ kbps}$; $T1 = 24 \text{ ch} = 1.544 \text{ Mbps}$; $E1 = 32 \text{ ch} = 2.048 \text{ Mbps}$
Bit stuffing	Insert 0 after five consecutive 1s; flag = 01111110
Parity	Detects odd-bit errors only
Checksum	Add words (wrap carry), complement; receiver gets 0
CRC	Append n zeros, XOR divide, remainder = CRC; remainder 0 = OK
Hamming code	$2^r \geq m + r + 1$; syndrome = error position
Hamming distance d	Detect d - 1 , correct [(d - 1) / 2]
Go-Back-N / Selective Repeat	Sender window $2^m - 1 / 2^{m-1}$

BPSC QUICK CHECK — Chapter 4

Circuit switching → dedicated path	Packet switching → header, no dedicated path
FDM → guard bands	TDM → time slots, digital
WDM → optical fibre	$T1 \rightarrow 1.544 \text{ Mbps}$ · $E1 \rightarrow 2.048 \text{ Mbps}$
Parity → odd errors only	CRC → strongest, XOR division
Checksum → complement of sum	Hamming → corrects single-bit errors

Traps: circuit vs packet · FDM vs TDM · odd vs even errors in parity · CRC uses XOR not subtraction · Hamming d - 1 and (d - 1)/2.

Practice questions

● LEVEL 1 · BEGINNER

- **Q1.** Name the three switching techniques.
- **Q2.** Which switching technique reserves a dedicated path?
- **Q3.** What is multiplexing?
- **Q4.** Which multiplexing is used on optical fibre?
- **Q5.** What is a parity bit?
- **Q6.** Expand CRC.

● LEVEL 2 · EXAM

- **Q7.** Find the even-parity bit for the data 1101011.
- **Q8.** Give three differences between datagram and virtual-circuit packet switching.
- **Q9.** Six channels of 200 kHz each are multiplexed by FDM with 20 kHz guard bands between channels. Find the total bandwidth.
- **Q10.** How many voice channels and what data rate does a T1 line have?
- **Q11.** Why does simple parity fail to detect two bit errors?
- **Q12.** Apply bit stuffing to the data 0111110111.

● LEVEL 3 · BPSC / PGT

- **Q13.** A message of 8000 bits is sent over 4 links of 1 Mbps each. Find the time with message switching and with packet switching using 1000-bit packets (ignore propagation and queuing).
- **Q14.** Find the 4-bit checksum for the words 6, 10, 4, 3.
- **Q15.** Find the CRC for data 1101 with generator 101. Write the transmitted codeword.
- **Q16.** How many check bits are needed for 11 data bits in a Hamming code?
- **Q17.** Find the Hamming distance between 1011001 and 1001101. If a code has $d_{\min} = 5$, how many errors can it detect and correct?
- **Q18.** A Hamming (7,4) word 1000101 (positions 1 to 7, even parity) is received. Find the error position and the corrected word.

Answers

Q	Solution
1	Circuit switching, message switching, packet switching.
2	Circuit switching.
3	Sending many signals together over one shared link, using a MUX at the sender and a DEMUX at the receiver.
4	WDM (wavelength division multiplexing), including DWDM.
5	An extra bit added to make the total number of 1s even (even parity) or odd (odd parity) so that errors can be detected.
6	Cyclic Redundancy Check.
7	1101011 has five 1s (odd), so the even-parity bit = 1 . Sent: 11010111.
8	Datagram: no setup, each packet routed separately, may arrive out of order, full address in each packet. Virtual circuit: path set up first, same path for all packets, arrive in order, short VC number in header.
9	$6 \times 200 + 5 \times 20 = 1200 + 100 = \mathbf{1300 \text{ kHz}}$ (5 guard bands between 6 channels).
10	24 voice channels, 1.544 Mbps ($24 \times 8 + 1 = 193$ bits per 125 μs).
11	Two flipped bits change the count of 1s by an even amount (or leave it the same), so the parity still looks correct.
12	Data 0 1 1 1 1 0 1 1 1: after five 1s insert 0 → 01111100111 .
13	Message switching: $4 \times 8000 \div 10^6 = \mathbf{32 \text{ ms}}$. Packet switching: $N = 8$, $T = (4 + 8 - 1) \times 1000 \div 10^6 = \mathbf{11 \text{ ms}}$.
14	Sum = $6 + 10 + 4 + 3 = 23 = 10111$. Wrap: $0111 + 1 = 1000 = 8$. Checksum = $15 - 8 = \mathbf{7}$. (Check: $23 + 7 = 30 = 11110 \rightarrow 1110 + 1 = 1111 \rightarrow$ complement 0 ✓.)
15	Generator 101 (degree 2), dividend 110100. $110 \oplus 101 = 011 \rightarrow$ rem 11, next bit 1 $\rightarrow 111 \oplus 101 = 010 \rightarrow$ rem 10, next bit 0 $\rightarrow 100 \oplus 101 = 001 \rightarrow$ rem 01, next bit 0 $\rightarrow 010 \rightarrow$ rem 10. CRC = 10 . Codeword = 110110 .
16	$2^r \geq 11 + r + 1$. $r = 3$: $8 \geq 15$ no. $r = 4$: $16 \geq 16$ yes. So 4 check bits.
17	$1011001 \oplus 1001101 = 0010100$, which has two 1s, so the distance = 2 . With $d_{\min} = 5$: detects $5 - 1 = \mathbf{4}$ errors, corrects $\lfloor (5 - 1) / 2 \rfloor = \mathbf{2}$ errors.
18	Bits at positions 1 to 7: 1,0,0,0,1,0,1. $c_1(1,3,5,7) = 1,0,1,1 \rightarrow$ odd $\rightarrow 1$. $c_2(2,3,6,7) = 0,0,0,1 \rightarrow$ odd $\rightarrow 1$. $c_4(4,5,6,7) = 0,1,0,1 \rightarrow$ even $\rightarrow 0$. Syndrome $c_4 c_2 c_1 = 011 = 3$, so position 3 is wrong. Corrected word = 1010101 .

REMEMBER

Next: Chapter 5 — OSI and TCP/IP Models (the seven OSI layers and their jobs, TCP/IP four layers, encapsulation, comparison of the two models).

Roadmap: Ch 6 Network Protocols (HTTP, FTP, SMTP, POP3, TELNET, PPP, VoIP, DNS) · Ch 7 IP Addressing and Subnetting · Ch 8 Web Services · Ch 9 Network Security · Ch 10 Mobile Telecom, Cloud and IoT.